

AI Hardware Design League (AI-HDL) Webinar

3: Technical Toolchain & Design Phase 1

Walkthrough

Video Source: [Webinar 3 Recording](#)

Session Lead: Harish Kumar (Technical Mentor)

1. Context & The "Guest Room" Analogy

Harish opens by grounding the technical work in a specific analogy to clarify the project scope.

- **The House (CPU):** You are provided with a fully functional RISC-V Core (specifically the `TinyQV` implementation). You do **not** need to build a CPU.
- **The Guest Room (Peripheral):** Your task is to build a specific room attached to the house.
- **The Hallway (Wrapper):** The connection between your room and the house is the `tt_wrapper`.
- **The Rule:** You primarily modify `peripheral.v`. You must ensure it talks to the CPU correctly through the wrapper.

Speaker's Insight: "We are focusing on modifying and optimizing the code, not rebuilding the complex CPU logic. Think of it as adding an accelerator or encryption engine to an existing system."

2. The AI-Assisted Workflow: A "Live" Co-Pilot Strategy

A defining feature of this webinar was Harish's refusal to rely on memorized commands. Instead, he demonstrated a **"Context-Injection" strategy** using Google Gemini. He treated the AI not just as a search engine, but as an active engineer pairing on the project.

A. The Core Philosophy: "Don't Memorize, Generate"

Harish emphasized that modern hardware design involves complex toolchains (Yosys, OpenLANE, Docker) with hundreds of dependencies. Memorizing every `apt-get` install string is inefficient.

- **Speaker's Method:** "I stick with using AI to help me with tool installation, fixing code, and generating scripts."
- **The Rule:** If you don't know the command, describe your *intent* and your *environment* to the AI.

B. Use Case 1: Installation & Dependencies (The "Scraper" Approach)

Instead of hunting through GitHub ReadMEs for Linux dependencies, Harish used the AI to aggregate them.

- **The Action (Timestamp ~10:20):** When installing Yosys, Harish didn't type the dependencies manually.
- **The Prompt Strategy:** He pasted the link to the Yosys GitHub repository or the specific error message into Gemini.
- **The AI Output:** Gemini generated a single, massive `sudo apt-get install...` block containing `clang`, `bison`, `flex`, `libreadline`, etc.
- **The Workflow:**
 1. Ask AI: "What are the prerequisites for building Yosys on Ubuntu?"
 2. Copy the code block.
 3. Paste into WSL terminal.
 4. *Crucial Step:* If an error occurs (e.g., "package not found"), paste the **exact error** back into Gemini to get the corrected package name for your specific Linux version.

C. Use Case 2: Script Generation (Synthesis)

Writing TCL scripts for Yosys can be syntax-heavy. Harish used AI to write the logic for him.

- **The Scenario (Timestamp ~36:15):** He needed a `synth.py` script to tell Yosys which files to read and what the top module was.
- **The Context-Injection Prompt:**

"I want to run synthesis. Here is my list of files: `peripheral.v`, `tt_wrapper.v`. The top module is `tt_wrapper`. Write a Yosys script for this."

- **The AI Output:**

The AI generated a perfect script:

```
read_verilog -sv peripheral.v
read_verilog -sv tt_wrapper.v
hierarchy -top tt_wrapper
synth -top tt_wrapper
stat
```

- **Why this matters:** Harish didn't need to remember if the command was `read_verilog` or `load_verilog`. He provided the *intent* ("run synthesis") and the *variables* (file names), and the AI handled the syntax.

D. Use Case 3: Configuration for Physical Design (OpenLANE)

OpenLANE requires a strict `config.json` file. One typo here breaks the entire flow.

- **The Scenario (Timestamp ~43:00):** He needed to tell OpenLANE about the clock speeds and file locations.
- **The Prompt:** He provided the AI with the paths to his files inside the Docker container.
- **The "Human-in-the-Loop" Correction:**
Harish noted that AI makes pathing errors. When Gemini generated the JSON, Harish paused to verify:
 - *AI suggestion:* `"dir/peripheral.v"`
 - *Harish's Correction:* He ensured the paths matched the *actual* directory structure in his WSL environment (`./designs/DP1/src/`).
 - **Takeaway:** Trust the AI for syntax (`"DESIGN_NAME": "..."`), but **verify the data** (file paths and clock periods).

E. Use Case 4: The Debugging Loop

Harish explicitly mentioned (Timestamp ~39:35) that if the terminal throws an error, you should not stare at it.

1. **Copy** the error message (e.g., `ERROR: module tt_wrapper not found`).
2. **Paste** it into the LLM.
3. **Action:** The LLM will usually tell you, "You forgot to include the file path in the `read_verilog` command," allowing you to fix the script in seconds rather than minutes.

F. Required Deliverable: Prompt Logs

Harish reiterated that this workflow isn't just a tip—it's a **requirement**.

- **Participant Action:** You must save these chat logs (like the one Harish had open on the left side of his screen throughout the demo).
- **Submission:** These logs serve as proof of your work process and are part of the grading rubric to evaluate how effectively you leveraged AI to solve engineering problems.

3. Step-by-Step Environment Setup

Note: This walkthrough assumes a Windows machine using WSL, as demonstrated by the speaker.

Step A: Windows Subsystem for Linux (WSL)

Since EDA tools generally don't run natively on Windows, Harish sets up a Linux container.

1. **Open PowerShell** as Administrator.
2. **Command:** `wsl --install`
 - *Speaker Note:* "This defaults to Ubuntu, which is what we want. It creates a containerized Linux OS inside Windows."
3. **Post-Install:** After restarting, open the "Ubuntu" app.
4. **User Setup:** Create a username (e.g., `harry`) and password.
 - *Critical:* Remember this password. You will need it for every `sudo` command.

Step B: Setting up the Project Directory

Harish emphasizes keeping a clean workspace.

1. **Create Directory:**

```
mkdir AI-HDL
cd AI-HDL
```

2. **Clone Base Repository:**

```
git clone https://github.com/MichaelBell/tinyqv.git
```

- *Context:* This repo contains the RISC-V core and the `peripheral.v` template you will edit.

4. The Toolchain Installation (Live Demo)

Tool 1: Yosys (Synthesis)

Objective: Turn Verilog code into logic gates.

1. **AI Interaction:** Harish pastes the Yosys GitHub link into Gemini and asks for prerequisites.
2. **Install Dependencies:**

```
sudo apt-get install build-essential clang bison flex \
libreadline-dev gawk tcl-dev libffi-dev git \
```

```
graphviz xdot pkg-config python3 libboost-system-dev \
libboost-python-dev libboost-filesystem-dev zlib1g-dev
```

3. Build Yosys:

```
git clone https://github.com/YosysHQ/yosys.git
cd yosys
make config-clang
make
sudo make install
```

- *Speaker Note:* "This compilation takes time (10-15 mins). It depends on your internet and CPU speed."

4. Verify: `yosys --version`

Tool 2: Verilator (Simulation)

Objective: High-speed verification.

1. **AI Interaction:** Harish asks Gemini, "How do I install Verilator on Ubuntu?"

2. Install Dependencies:

```
sudo apt-get install git help2man perl python3 make autoconf g++ flex
sudo apt-get install libgoogle-perftools-dev numactl perl-doc
```

3. Build Verilator:

```
git clone https://github.com/verilator/verilator
cd verilator
autoconf
./configure
make
sudo make install
```

4. Verify: `verilator --version`

Tool 3: KLayout (Layout Viewer)

Objective: Viewing the final GDSII (chip blueprint).

1. Command:

```
sudo apt-get install klayout
```

2. **Launch:** Type `klayout` in the terminal.

- *Visual:* A GUI window opens. Harish notes, "This is not a terminal tool; it's a visualizer."

Tool 4: OpenROAD (Physical Design)

Objective: Floorplanning, placement, and routing.

1. Clone Recursive:

```
git clone --recursive https://github.com/The-OpenROAD-Project/OpenROAD
cd OpenROAD
```

- *Speaker Note:* "The `--recursive` flag is vital because OpenROAD relies on many sub-modules."

2. Install Dependencies Script:

```
sudo ./etc/DependencyInstaller.sh
```

3. Build:

```
mkdir build
cd build
cmake ..
make
```

Tool 5: OpenLANE (The "Manager")

Objective: Automating the flow from RTL to GDSII using Docker.

1. Install Docker:

- Harish navigates to the Docker website.
- **Windows Users:** Install "Docker Desktop."
- **Linux Users:** `sudo apt-get install docker.io`

2. Clone OpenLANE:

```
git clone https://github.com/The-OpenROAD-Project/OpenLane.git
cd OpenLane
```

3. Initialize Container:

```
make mount
```

- **Context:** This command pulls the docker image and drops you into a shell inside the container. The terminal prompt changes to verify you are inside the tool environment.
-

5. Design Phase 1: The "Live" Project Run

Harish moves from installation to execution, demonstrating exactly how a team should tackle DP#1.

Phase 1A: Setup & File Management

1. **Create Design Folder:** He creates a folder named `DP1` inside his directory.
2. **Copy Files:** He copies `peripheral.v` (the module) and `tt_wrapper.v` (the wrapper) from the `tinyqv` folder into `DP1`.
 - *Speaker Tip:* "Always keep your design files separate from the tool directories initially."

Phase 1B: Synthesis with Yosys (AI-Guided)

Harish creates a synthesis script to check if the design is valid.

1. **The Prompt:** He lists his files (`peripheral.v` , `tt_wrapper.v`) in Gemini and asks: "Write a Yosys script to synthesize these. The top module is `tt_wrapper`."
2. **The Resulting Script (`synth.py`):**

Harish uses `nano synth.py` to paste the AI-generated code:

```
read_verilog -sv peripheral.v
read_verilog -sv tt_wrapper.v
hierarchy -top tt_wrapper
synth -top tt_wrapper
stat
```

3. Execution:

```
yosys synth.py
```

4. **Output Analysis:** The terminal prints a table of "Printing Statistics."
 - *Speaker Insight:* "If you see numbers here—cells, flip-flops—you are good. If you see 'ERROR: Module not found', check your file paths."

Phase 1C: PPA Analysis with OpenLANE (The Big Run)

This is the core competition task: getting Power, Performance, and Area metrics.

1. Migrate to OpenLANE:

Harish copies the `DP1` folder (containing his Verilog files) into the `OpenLane/designs/` directory.

- Path: `~/OpenLane/designs/DP1/`

2. Configuration (`config.json`):

He creates a configuration file required by OpenLANE. Again, he uses AI to format it correctly based on the OpenLANE documentation.

- Key Settings:
 - `DESIGN_NAME` : "tt_wrapper"
 - `VERILOG_FILES` : Lists the paths to `peripheral.v` and `tt_wrapper.v`.
 - `CLOCK_PERIOD` : "10.0"

3. Running the Flow:

Inside the `OpenLane` directory:

```
make mount    # Enters Docker
./flow.tcl -design DP1
```

- Visual: The terminal scrolls rapidly with logs (Synthesis -> Floorplan -> Placement -> CTS -> Routing).
- Time: It took about 5-7 minutes in the video.

4. Success Condition:

The terminal displays: `[SUCCESS]: Flow Completed Without Fatal Errors.`

Phase 1D: Finding the Results (The "Score")

Harish demonstrates where to find the data needed for the competition submission.

1. Navigate to Reports:

He exits Docker and goes to:

`OpenLane/designs/DP1/runs/<date_timestamp>/reports/`.

2. The Metrics File: He opens `metrics.csv`.

3. Key Columns Explained:

- `die_area / Total Area` : The physical size. Smaller is better.
- `Total Internal Power` : Energy consumed. Lower is better.
- `wns (Worst Negative Slack)`:
 - Speaker Warning: "This is your timing score. If this number is negative (e.g., -2.5), your design failed timing. You must optimize it until this number is positive."

4. Viewing the Chip:

He navigates to the `results/final/gds/` folder and runs:

```
klayout tt_wrapper.gds
```

- *Visual:* The screen shows the complex, colorful layers of the physical chip.
-

6. Closing Q&A & Key Takeaways

- **Prompt Logs are Mandatory:** You must save the conversations you have with ChatGPT/Gemini (like Harish did when asking for the Yosys script) and submit them.
- **Team Formation:** If you are solo, the organizers will pair you up after registration closes (Dec 19).
- **Computer Specs:** "You don't need a supercomputer." The demo was run on a standard laptop. 8GB RAM is the floor; 16GB allows Docker to breathe easier.
- **Timeline:**
 - **Now - Jan 15:** Install tools, play with `peripheral.v`.
 - **Jan 15:** Competition Kickoff.
- **Discord:** All specific errors ("My Docker isn't starting") should be posted in the technical support channels on Discord.